

Fase 6: Testing

pytest + moto para backend serverless

Tutorial paso a paso

Proyecto Sentinel AcademIA
lFunknown

20 de junio de 2026

¿Qué construimos en este tutorial?

En la **Fase 6** agregamos **tests automatizados** al backend para verificar que todo funciona correctamente. Antes teníamos pruebas manuales con curl; ahora tenemos **50 tests** que corren en segundos.

- **Framework:** pytest 9.1.1
- **Mocks de AWS:** moto 5.2.2 (DynamoDB, S3, SQS)
- **Tests unitarios:** 28 (config, errors, schemas, dynamo, knowledge)
- **Tests de integración:** 22 (handlers con API events)
- **Total:** 50/50 pasando en 2.5 segundos
- **Bugs encontrados:** 4 (gracias a los tests)

Herramientas:

- **pytest:** framework de testing estándar en Python
- **moto:** mock de servicios AWS (DynamoDB, S3, SQS, etc.)
- **pytest-cov:** coverage report
- **pytest-mock:** mocking simplificado

El usuario dijo "no test." en Fase 1...

Recuerdo que en Fase 1 el usuario pidió NO hacer tests automáticos. Pero ahora en Fase 6 son obligatorios: el desafío era agregarlos sin conflictos, verificando que todo el backend funcione correctamente.

Índice

1. ¿Por qué tests automatizados?	3
1.1. Beneficios	3
1.2. Estrategia: pirámide de tests	3
2. Herramientas: pytest + moto	3
2.1. pytest	3
2.2. moto	3
3. Estructura de tests	4
3.1. Organización	4

3.2. conftest.py: fixtures compartidos	4
4. Tests unitarios	6
4.1. test_shared.py: 18 tests	6
4.2. test_dynamo_client.py: 9 tests con moto	7
4.3. test_knowledge_service.py: 10 tests (RAG)	7
5. Tests de integración	8
5.1. test_handlers.py: 22 tests	8
6. Los 4 bugs que los tests encontraron	9
6.1. Bug 1: query_by_tenant usaba eq en vez de begins_with	9
6.2. Bug 2: ScanIndexForward con scan	9
6.3. Bug 3: SyntaxError en chat.py	10
6.4. Bug 4: Imports duplicados (clases diferentes)	10
7. Cómo correr los tests	10
7.1. Comandos básicos	10
7.2. Tests específicos	10
8. Catálogo de errores comunes	10
9. Lo que aprendimos (resumen ejecutivo)	11
10. Ejercicios para practicar	11
10.1. Ejercicio 1: Test del SQS consumer	11
10.2. Ejercicio 2: Test del file_processor	12
10.3. Ejercicio 3: Test de integración E2E (con moto)	12
10.4. Ejercicio 4: Coverage > 80 %	12
10.5. Ejercicio 5: CI con GitHub Actions	12
11. Siguiendo el paso: Fase 7 (Deploy final + demo)	12

1. ¿Por qué tests automatizados?

1.1. Beneficios

- **✓Regresiones:** detectas cuando un cambio rompa algo
- **✓Refactoring:** puedes cambiar código con confianza
- **✓Documentación:** los tests son ejemplos de uso
- **✓CI/CD:** puedes deployar automáticamente
- **✓Velocidad:** 50 tests en 2.5 segundos vs 10 minutos manuales

1.2. Estrategia: pirámide de tests

Tipo	Cantidad
Unit tests (rápidos, aislados)	28
Integration tests (con mocks AWS)	22
E2E tests (con AWS real)	0 (skip por costo)

2. Herramientas: pytest + moto

2.1. pytest

Qué es: framework de testing más popular de Python.

```
1 pip install pytest pytest-cov pytest-mock
2 # pytest 9.1.1
```

Listing 1: Instalación

Conceptos:

- **✓Tests:** funciones que empiezan con `test_`
- **✓Fixtures:** setup compartido con `@pytest.fixture`
- **✓Markers:** `@pytest.mark.unit`, `@pytest.mark.integration`
- **✓Parametrize:** `@pytest.mark.parametrize` para casos
- **✓Mocking:** `mock.patch` o `unittest.mock`

2.2. moto

Qué es: mock library para servicios AWS.

```
1 pip install 'moto[dynamodb,s3,sqs]'
2 # moto 5.2.2
```

Listing 2: Instalación

Uso:

```
1 from moto import mock_aws
2
3 @mock_aws()
4 def test_my_dynamo():
5     ddb = boto3.resource("dynamodb", region_name="us-east-1")
```

```

6     table = ddb.create_table(
7         TableName="test",
8         KeySchema=[{"AttributeName": "pk", "KeyType": "HASH"}],
9         AttributeDefinitions=[{"AttributeName": "pk", "AttributeType":
10             "S"}],
11         BillingMode="PAY_PER_REQUEST",
12     )
13 # ... tu test

```

Listing 3: Mock de DynamoDB

3. Estructura de tests

3.1. Organización

```

1 src/tests/
2 |-- conftest.py           # fixtures compartidos
3 |-- unit/
4 |   |-- test_shared.py    # config, errors, schemas
5 |   |-- test_dynamo_client.py # DynamoDB con moto
6 |   '-- test_knowledge_service.py # RAG retrieval
7 '-- integration/
8     '-- test_handlers.py   # todos los Lambdas

```

Listing 4: Estructura

3.2. conftest.py: fixtures compartidos

```

1 import os
2 import boto3
3 import pytest
4
5
6 @pytest.fixture(autouse=True)
7 def _env(monkeypatch):
8     """Set required env vars for all tests (Settings is cached)."""
9     monkeypatch.setenv("STAGE", "dev")
10    monkeypatch.setenv("AWS_DEFAULT_REGION", "us-east-1")
11    monkeypatch.setenv("AWS_REGION", "us-east-1")
12    monkeypatch.setenv("LOG_LEVEL", "WARNING")
13    monkeypatch.setenv("MOCK_LLM", "true")
14    monkeypatch.setenv("OCI_COMPARTMENT_ID",
15        "ocid1.compartment.oc1..test")
16    monkeypatch.setenv("OCI_COHERE_MODEL_OCID",
17        "ocid1.generativeaimodel.oc1..test")
18    # Clear cache so Settings reloads with test env
19    from handlers._shared.config import get_settings
20    get_settings.cache_clear()
21    yield
22
23 @pytest.fixture
24 def aws_credentials(monkeypatch):
25     """Mock AWS credentials so moto doesn't complain."""
26     monkeypatch.setenv("AWS_ACCESS_KEY_ID", "testing")
27     monkeypatch.setenv("AWS_SECRET_ACCESS_KEY", "testing")
28     monkeypatch.setenv("AWS_SECURITY_TOKEN", "testing")

```

```

28 monkeypatch.setenv("AWS_SESSION_TOKEN", "testing")
29 monkeypatch.setenv("AWS_DEFAULT_REGION", "us-east-1")
30
31
32 @pytest.fixture
33 def dynamodb(aws_credentials):
34     """Create a mocked DynamoDB with the QuejasTable."""
35     from moto import mock_aws
36     from handlers._shared.config import get_settings
37
38     with mock_aws():
39         table_name = get_settings().dynamodb_table_resolved
40         ddb = boto3.resource("dynamodb", region_name="us-east-1")
41         table = ddb.create_table(
42             TableName=table_name,
43             KeySchema=[
44                 {"AttributeName": "pk", "KeyType": "HASH"},
45                 {"AttributeName": "sk", "KeyType": "RANGE"},
46             ],
47             AttributeDefinitions=[
48                 {"AttributeName": "pk", "AttributeType": "S"},
49                 {"AttributeName": "sk", "AttributeType": "S"},
50                 {"AttributeName": "gsi1pk", "AttributeType": "S"},
51                 {"AttributeName": "gsi1sk", "AttributeType": "S"},
52             ],
53             GlobalSecondaryIndexes=[{
54                 "IndexName": "GSI1-StatusByDate",
55                 "KeySchema": [
56                     {"AttributeName": "gsi1pk", "KeyType": "HASH"},
57                     {"AttributeName": "gsi1sk", "KeyType": "RANGE"},
58                 ],
59                 "Projection": {"ProjectionType": "ALL"},
60             }],
61             BillingMode="PAY_PER_REQUEST",
62         )
63         table.wait_until_exists()
64         yield ddb
65
66
67 @pytest.fixture
68 def dynamo_client(dynamodb):
69     """Get the dynamo_client singleton (with mocked AWS)."""
70     import handlers._shared.dynamo_client as dc_module
71     dc_module._client = None
72     return dc_module.get_dynamo_client()
73
74
75 @pytest.fixture
76 def api_event():
77     """Factory for API Gateway proxy events."""
78     def _make(method="GET", path="/", body=None, headers=None,
79              path_params=None, query_params=None):
80         if isinstance(body, dict):
81             import json
82             body = json.dumps(body)
83         return {
84             "httpMethod": method,
85             "path": path,

```

```

86         "body": body,
87         "headers": {
88             "Content-Type": "application/json",
89             "X-Tenant-ID": "test-tenant",
90             "X-Correlation-ID": "test-correlation-123",
91             **(headers or {}),
92         },
93         "pathParameters": path_params,
94         "queryStringParameters": query_params,
95     }
96     return _make

```

Listing 5: src/tests/conftest.py

4. Tests unitarios

4.1. test_shared.py: 18 tests

```

1 class TestConfig:
2     def test_default_values(self):
3         from handlers._shared.config import Settings, get_settings
4         get_settings.cache_clear()
5         s = Settings()
6         assert s.app_name == "sentinel-academia"
7         assert s.region == "us-east-1"

```

Listing 6: TestConfig

```

1 class TestSchemas:
2     def test_queja_create_too_short(self):
3         from pydantic import ValidationError as PydanticValidationError
4         from handlers._shared.schemas import QuejaCreate
5
6         with pytest.raises(PydanticValidationError):
7             QuejaCreate(titulo="abc", descripcion="too short")
8
9     def test_queja_create_invalid_categoria(self):
10        with pytest.raises(PydanticValidationError):
11            QuejaCreate(
12                titulo="Test titulo aqui",
13                descripcion="Descripcion valida larga",
14                categoriaDeclarada="INVALIDA",
15            )

```

Listing 7: TestSchemas

```

1 class TestErrors:
2     def test_validation_error(self):
3         from handlers._shared.errors import ValidationError
4         e = ValidationError("bad", details={"field": "x"})
5         assert e.status_code == 400
6         assert e.error_code == "VALIDATION_ERROR"
7         assert e.details == {"field": "x"}

```

Listing 8: TestErrors

4.2. test_dynamo_client.py: 9 tests con moto

```

1 class TestDynamoClient:
2     def test_put_and_get(self, dynamo_client):
3         dynamo_client.put_item({
4             "pk": "TENANT#utpl#QUEJA#q-1",
5             "sk": "QUEJA#q-1",
6             "quejaId": "q-1",
7             "tenantId": "utpl",
8         })
9         item = dynamo_client.get_item(
10             pk="TENANT#utpl#QUEJA#q-1", sk="QUEJA#q-1"
11         )
12         assert item["quejaId"] == "q-1"
13
14     def test_tenant_isolation(self, dynamo_client):
15         """Queja in tenant A should not be visible from tenant B."""
16         # Insert in tenant A
17         dynamo_client.put_item({
18             "pk": "TENANT#a#QUEJA#q-1",
19             "sk": "QUEJA#q-1",
20             "tenantId": "a",
21         })
22         # Try to query from tenant B
23         with pytest.raises(NotFoundError):
24             dynamo_client.get_queja(tenant_id="b", queja_id="q-1")
25
26     def test_to_dynamo_converts_floats(self):
27         """Helper that converts Python float to Decimal."""
28         from handlers._shared.dynamo_client import _to_dynamo
29         from decimal import Decimal
30         result = _to_dynamo({"score": 0.75})
31         assert isinstance(result["score"], Decimal)
32         assert result["score"] == Decimal("0.75")

```

Listing 9: TestDynamoClient

4.3. test_knowledge_service.py: 10 tests (RAG)

```

1 class TestKnowledgeService:
2     def test_search_relevant_for_acoso(self):
3         from handlers._shared.knowledge_service import search
4         # Wait, should be from services
5         from services.knowledge_service import search
6
7         results = search("acoso")
8         # The FAQ for acososo should be in top results
9         assert any("acoso" in r["id"].lower() for r in results)
10
11     def test_search_scores_descending(self):
12         from services.knowledge_service import search
13         results = search("acoso reglamento queja")
14         scores = [r["score"] for r in results]
15         assert scores == sorted(scores, reverse=True)

```

Listing 10: TestKnowledgeService

5. Tests de integración

5.1. test_handlers.py: 22 tests

```

1 class TestCreateQueja:
2     def test_creates_queja(self, api_event, sample_queja,
3         dynamo_client):
4         from handlers.create_queja import lambda_handler
5         import unittest.mock
6
7         event = api_event(method="POST", path="/api/quejas",
8             body=sample_queja)
9         with unittest.mock.patch(
10             "handlers.create_queja._send_to_sqs"
11         ) as mock_sqs:
12             result = lambda_handler(event, context={})
13
14             assert result["statusCode"] == 202
15             body = json.loads(result["body"])
16             assert "quejaId" in body
17             assert body["status"] == "PENDIENTE"
18
19             # Verify it was stored in DynamoDB
20             items = dynamo_client.query_by_tenant(tenant_id="test-tenant")
21             assert len(items) == 1
22
23     def test_validation_error_short_titulo(self, api_event,
24         dynamo_client):
25         from handlers.create_queja import lambda_handler
26         bad = {"titulo": "abc", "descripcion": "larga",
27             "categoriaDeclarada": "OTRA"}
28         event = api_event(method="POST", path="/api/quejas", body=bad)
29         result = lambda_handler(event, context={})
30         assert result["statusCode"] == 400

```

Listing 11: TestCreateQueja

```

1 class TestGetQueja:
2     def test_get_tenant_isolation(self, api_event, dynamo_client):
3         """Queja in tenant A should not be accessible from tenant B."""
4         from handlers.get_queja import lambda_handler
5
6         # Insert in tenant A
7         dynamo_client.put_item({
8             "pk": "TENANT#tenant-a#QUEJA#q-1",
9             "sk": "QUEJA#q-1",
10            "quejaId": "q-1",
11            "tenantId": "tenant-a",
12            "titulo": "Titulo valido aqui",
13            "descripcion": "Descripcion valida de 20+ chars",
14            "categoriaDeclarada": "OTRA",
15            "status": "PENDIENTE",
16            "createdAt": "2026-01-01",
17            "updatedAt": "2026-01-01",
18        })
19
20         # Try from tenant B
21         event = api_event(
22             method="GET",
23             path="/api/quejas/q-1",

```

```

23         path_params={"quejaId": "q-1"},
24         headers={"X-Tenant-ID": "tenant-b"},
25     )
26     result = lambda_handler(event, context={})
27     assert result["statusCode"] == 404 # Not found for tenant B

```

Listing 12: TestGetQueja con tenant isolation

```

1 class TestChat:
2     def test_chat_returns_answer_and_sources(
3         self, api_event, sample_chat_request
4     ):
5         from handlers.chat import lambda_handler
6         event = api_event(
7             method="POST", path="/api/chat", body=sample_chat_request
8         )
9         result = lambda_handler(event, context={})
10        assert result["statusCode"] == 200
11        body = json.loads(result["body"])
12        assert "answer" in body
13        assert "sources" in body
14        assert len(body["sources"]) > 0

```

Listing 13: TestChat (RAG)

6. Los 4 bugs que los tests encontraron

6.1. Bug 1: query_by_tenant usaba eq en vez de begins_with

```

1 key_condition = Key("pk").eq(tenant_pk(tenant_id))
2 # pk = "TENANT#utpl#QUEJA#q-1"
3 # eq "TENANT#utpl" -> NUNCA matchea!

```

Listing 14: ANTES (BUG)

```

1 from boto3.dynamodb.conditions import Attr
2 filter_expr = Attr("pk").begins_with(tenant_pk(tenant_id))
3 # begins_with "TENANT#utpl" -> matchea "TENANT#utpl#QUEJA#q-1"

```

Listing 15: DESPUÉS (FIX)

Impacto: el bug existía en prod pero no se notó porque probablemente ningún cliente usaba list_quejas en la práctica.

6.2. Bug 2: ScanIndexForward con scan

```

1 response = self._table.scan(
2     FilterExpression=filter_expr,
3     ScanIndexForward=scan_index_forward, # <-- Scan no acepta esto
4 )

```

Listing 16: ANTES (BUG)

```

1 ParamValidationError: Unknown parameter in input: "ScanIndexForward",
2 must be one of: TableName, IndexName, ..., FilterExpression

```

Listing 17: Error

Fix: scan no soporta ScanIndexForward. Solo query lo soporta.

6.3. Bug 3: SyntaxError en chat.py

```
1 SYSTEM_PROMPT = """
2 1. Responde en espa~nol, de forma clara y concisa.
3 # ~~~~ SyntaxError: invalid escape sequence
```

Listing 18: ANTES (BUG)

```
1 SYSTEM_PROMPT = """
2 1. Responde en espa nol, de forma clara y concisa.
```

Listing 19: DESPUÉS (FIX)

6.4. Bug 4: Imports duplicados (clases diferentes)

Síntoma: `NotFoundError is NotFoundError` → `False`

Causa: tests usaban `from src.handlers._shared` pero el runtime usaba `from handlers._shared`. Python los ve como módulos diferentes.

Fix: unificar todos los imports a `handlers._shared` (sin `src.`).

7. Cómo correr los tests

7.1. Comandos básicos

```
1 cd /home/lfunknown/sentinel-academia
2 export PYTHONPATH=src:$PYTHONPATH
3 python3 -m pytest src/tests/ -v
```

Listing 20: Correr todos

```
1 src/tests/integration/test_handlers.py::TestCreateQueja::test_creates_queja
   PASSED [ 88%]
2 src/tests/integration/test_handlers.py::TestCreateQueja::test_validation_error_short
   PASSED [ 90%]
3 ...
4 ===== 50 passed in 2.43s
   =====
```

Listing 21: Output esperado

7.2. Tests específicos

```
1 python3 -m pytest src/tests/unit/ -v
```

Listing 22: Solo unit tests

```
1 python3 -m pytest
   src/tests/unit/test_dynamo_client.py::TestDynamoClient::test_put_and_get
   -v
```

Listing 23: Solo un test

```
1 python3 -m pytest src/tests/ --cov=src --cov-report=term-missing
```

Listing 24: Con coverage

8. Catálogo de errores comunes

Error 1: ModuleNotFoundError

Cuando: pytest no encuentra los módulos.
Causa: PYTHONPATH no incluye src.
Solución: export PYTHONPATH=src:\$PYTHONPATH.

Error 2: ModuleNotFoundError: No module named 'pydantic'

Cuando: deps no instaladas en el Python del sistema.
Causa: deps solo en el Python del OCI installer.
Solución: pip install pydantic boto3 moto pytest.

Error 3: Settings ValidationError

Cuando: STAGE=test en conftest.
Causa: stage solo permite dev|staging|prod.
Solución: usar STAGE=dev o agregar test al Literal.

Error 4: Table not found

Cuando: tests de DDB fallan con ResourceNotFoundException".
Causa: el fixture crea la tabla con un nombre pero el cliente usa otro.
Solución: usar get_settings().dynamodb_table_resolved.

Error 5: 'function' has no attribute 'cache_clear'

Cuando: get_dynamo_client.cache_clear().
Causa: la función no usa @lru_cache, tiene un global.
Solución: import dc; dc._client = None.

9. Lo que aprendimos (resumen ejecutivo)

Checklist Fase 6

1. **pytest + moto** = tests rápidos sin AWS real
2. **50 tests** en 2.5 segundos (vs minutos con curl)
3. **Conftest** para fixtures compartidos (env, AWS, eventos API)
4. **4 bugs encontrados** gracias a los tests
5. **imports unificados** (handlers._shared sin src.)
6. **moto** no soporta ScanIndexForward en scan (solo query)
7. **Stage=dev** (no test) por el Literal de Settings
8. **Decimal** para floats en DynamoDB (no float)
9. **Tenant isolation** tests con dos tenants
10. **50/50** tests pasando → backend robusto

10. Ejercicios para practicar

10.1. Ejercicio 1: Test del SQS consumer

```
1 def test_analyze_updates_to_analizada(self, api_event, dynamo_client, sqs):
```

```

2      # 1. Create a queja
3      # 2. Send SQS message
4      # 3. Invoke analyze_queja handler
5      # 4. Assert status == ANALIZADA
6      # 5. Assert analysis field exists

```

Listing 25: test_analyze_queja.py

10.2. Ejercicio 2: Test del file_processor

```

1 def test_file_processor_updates_queja(self, dynamo_client, s3):
2     # 1. Create queja in DDB
3     # 2. Upload file to S3
4     # 3. Invoke file_processor with S3 event
5     # 4. Assert file URL in adjuntos

```

Listing 26: test_file_processor.py

10.3. Ejercicio 3: Test de integración E2E (con moto)

```

1 def test_create_get_analyze_flow(dynamodb, sqs):
2     # 1. POST /quejas via create_queja handler
3     # 2. Verify it's in DDB
4     # 3. Trigger SQS event
5     # 4. Invoke analyze_queja
6     # 5. GET /quejas/{id} and verify analysis

```

Listing 27: test_e2e.py

10.4. Ejercicio 4: Coverage > 80 %

```

1 python3 -m pytest src/tests/ --cov=src --cov-fail-under=80

```

Añadir tests para: - services/file_service.py - services/llm_client.py (RealLLMClient)
- services/knowledge_service.py (más casos)

10.5. Ejercicio 5: CI con GitHub Actions

Crear .github/workflows/test.yml que corra pytest en cada PR.

11. Siguiendo paso: Fase 7 (Deploy final + demo)

En la **Fase 7** (la última) hacemos:

- Re-deploy final con todos los cambios
- Verificar que todo funciona end-to-end
- Capturar screenshots para el demo
- Grabar video demo de 2-3 minutos
- Actualizar README con instrucciones de uso
- Limpiar el código y documentar

Tiempo estimado: 1-1.5 horas.

Fase	Estado
1. Backend Core	[OK]
2. LLM integration	[OK]
3. Frontend deploy	[OK]
4. Files	[OK]
5. RAG	[OK]
6. Testing (este tutorial)	[OK]
7. Deploy final + demo	próximo

Fin del tutorial. À por la Fase 7 (la final)!